

DD: SUB-WF-RELOCATION-01 — Subscription Relocation

Developer implementation guide:

DD_SUB-WF-RELOCATION-01-Subscription_Relocation-DEV_IMPL_GUIDE-v1.1.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract

DD_SUB-WF-RELOCATION-01-Subscription_Relocation-v1.1.md.

Verdict: ■ Updated | **Wave:** 1 | **Group I:** Subscription Management | **Version:** v1.1 Operation that transitions an ACTIVE subscription's `homepass_id` while keeping its Package unchanged. Used when: (a) customer is physically moving to a new address that the operator serves (same Package, different HomePass); (b) operator-driven address correction (rare). Owns: the 2-trigger taxonomy, target-HomePass validation, equipmentPath + disconnectMode taxonomies, effectiveTiming taxonomy with cancellable scheduled-future option, the integration surface to WO-01 (SHIFTING flow) for physical work, the per-operator config catalog, the trigger + cancel REST endpoints, the rich domain Kafka events.

Physical work delegated to WO-01. When the customer needs tech dispatch (the default), this workflow creates ONE Work Order with `kind=SHIFTING` via `POST /api/work-orders`. The WO module's SHIFTING flow handles the entire physical operation internally as a multi-phase BPMN (DISCONNECT → RECONNECT). This workflow parks on `WorkOrderFinalized` for the WO completion. The WO-01 contract surface is documented in DD_WO-01-CONTRACT-FOR-SUB-WF.

Customer-self-move path. When `disconnectMode = CUSTOMER_SELF_MOVE`, no SHIFTING WO is created — the customer self-handles disconnection, transport, and reconnection of their own equipment. This workflow runs the source-side teardown + target-side activation steps directly without a tech dispatch. **Atomic-switch service continuity:** subscription stays ACTIVE on the source HomePass until the target activate succeeds. `PENDING_RELOCATION` is entered only during the commit window. If the customer wants a service gap, they request a separate PAUSE. **What changed from v1.0:** - **Replaced the made-up "two-WO orchestration" with the real WO-01 SHIFTING flow integration** per DD_WO-01-CONTRACT-FOR-SUB-WF. One WO per relocation, multi-phase mechanism handled by WO-01. - **Replaced fabricated message catches** (disconnection-completed, installation-completed) with the real framework events (`WorkOrderPhaseTransitioned`, `WorkOrderFinalized`). - **Removed fabricated config columns** (`disconnect_wo_timeout_days`, `install_wo_timeout_days`). WO timeouts come from `wo_kind_config.default_timeout_days` for `kind=SHIFTING`. - **Removed fabricated worker topics** (`wo-create-disconnect`, `wo-create-install`). Single worker `wo-create-shifting` creates the WO via the framework's WO creation endpoint. - **Operator-neutral framing.** Per-operator concerns (`job_type_code` values, BPMN file names, equipment recovery policy, account-status code values) live in WO-01 FLOW satellites + operator catalogs. This workflow speaks the integration surface only.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/04_subscription_workflows/DD_SUB-WF-RELOCATION-01-Subscription_Relocation-v1.1.md
Version	v1.1
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	rules.subscription.relocation.<operator>

4. Runtime Contracts To Implement

4.1 APIs

- DELETE /api/subscriptions/{subscription_id}/pending-relocation/{operation_id}
- DELETE /pending-relocation/{operationId}
- POST /api/subscriptions/{subscription_id}/relocate
- POST /api/work-orders

4.2 Tables

- subscription_relocation_config

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- bil01-emit-intent
- disconnection-completed
- external-cancel-requested
- ful-call-activate
- ful-call-claim-homepass
- ful-call-release-homepass
- ful-call-suspend
- installation-completed
- notification-send
- payment-confirmed
- wo-cancel
- wo-create-disconnect
- wo-create-install
- wo-create-shifting

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.

- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- SubscriptionRelocated
- SubscriptionRelocationCancelled
- SubscriptionRelocationRejected
- SubscriptionRelocationScheduled
- WorkOrderCancelled
- WorkOrderFinalized
- WorkOrderPhaseTransitioned
- WorkOrderTimedOut

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Operation that transitions an ACTIVE subscription's homepass_id while keeping its Package unchanged. Used when: (a) customer is physically moving to a new address that the operator serves (same Package, different HomePass); (b) operator-driven address correction (rare). Owns: the 2-trigger taxonomy, target-HomePass validation, equipmentPath + disconnectMode taxonomies, effectiveTiming taxonomy with cancellable scheduled-future option, the integration surface to WO-01 (SHIFTING flow) for physical work, the per-operator config catalog, the trigger + cancel REST endpoints, the rich domain Kafka events.

Physical work delegated to WO-01. When the customer needs tech dispatch (the default), this workflow creates ONE Work Order with kind=SHIFTING via POST /api/work-orders. The WO module's SHIFTING flow handles the entire physical operation internally as a multi-phase BPMN (DISCONNECT → RECONNECT). This workflow parks on WorkOrderFinalized for the WO completion. The WO-01 contract surface is documented in DD_WO-01-CONTRACT-FOR-SUB-WF.

Customer-self-move path. When disconnectMode = CUSTOMER_SELF_MOVE, no SHIFTING WO is created — the customer self-handles disconnection, transport, and reconnection of their own equipment. This workflow runs the source-side teardown + target-side activation steps directly without a tech dispatch.

Atomic-switch service continuity: subscription stays ACTIVE on the source HomePass until the target activate succeeds. PENDING_RELOCATION is entered only during the commit window. If the customer wants a service gap, they request a separate PAUSE.

What changed from v1.0:

- **Replaced the made-up "two-WO orchestration" with the real WO-01 SHIFTING flow integration** per DD_WO-01-CONTRACT-FOR-SUB-WF. One WO per relocation, multi-phase mechanism handled by WO-01.
- **Replaced fabricated message catches** (disconnection-completed, installation-completed) with the real framework events (WorkOrderPhaseTransitioned, WorkOrderFinalized).
- **Removed fabricated config columns** (disconnect_wo_timeout_days, install_wo_timeout_days). WO timeouts come from wo_kind_config.default_timeout_days for kind=SHIFTING.
- **Removed fabricated worker topics** (wo-create-disconnect, wo-create-install). Single worker wo-create-shifting creates the WO via the framework's WO creation endpoint.
- **Operator-neutral framing.** Per-operator concerns (job_type_code values, BPMN file names, equipment recovery policy, account-status code values) live in WO-01 FLOW satellites + operator catalogs. This workflow speaks the integration surface only.

Glossary

Term	Meaning
Relocation	Change of subscription's homepass_id while keeping package_ref unchanged. Distinct from MIGRATION (changes both HomePass and Package) and from UPGRADE/DOWNGRADE (changes Package only).
Source HomePass	Current homepass_id snapshot at relocation time.
Target HomePass	The HomePass the subscription is moving to. Must support every service family required by the current Package and have status sellable+active.
equipmentPath	Resolved per request: KEEP_EXISTING (re-home existing equipment at target) or NEW_INSTALL (install fresh equipment, recover old). Auto-inferred from source/target network-tech compatibility; request override allowed.
disconnectMode	Resolved per request: TECH_DISPATCH (standard — workflow creates a SHIFTING WO and tech handles physical work) or CUSTOMER_SELF_MOVE (no WO; customer handles physical move themselves). NEW_INSTALL always forces TECH_DISPATCH.
effectiveTiming	When the relocation commits: IMMEDIATE / END_OF_CURRENT_CYCLE / SCHEDULED_AT (the default — customers know the move date in advance).
Pending-future relocation	Scheduled relocation in wait state. BPMN parked on Camunda timer + subscription_operation.current_state = SCHEDULED. Cancellable via DELETE.

Term	Meaning
Atomic-switch	Commit-window sequence: FUL-04 SUSPEND at source → FUL-07 RELEASE on source HomePass → [SHIFTING WO via WO-01, OR self-move acknowledgement] → FUL-03 ACTIVATE at target with equipment bindings from the WO event (or pre-existing for self-move) → FUL-07 CLAIM on target HomePass → commit master fields. Subscription stays ACTIVE on source until the entire sequence succeeds.
SHIFTING WO	A Work Order with kind=SHIFTING created via WO-01's POST /api/work-orders endpoint. The WO module's internal multi-phase BPMN handles disconnect + reconnect autonomously. Contract surface in DD_WO-01-CONTRACT-FOR-SUB-WF.

1. What this is

The relocation operation. Triggered by: a customer requesting an address change while keeping their current Package (CUSTOMER_REQUESTED_RELOCATION), or an admin override for ops scenarios (ADMIN_RELOCATION).

Outcomes (at actual commit time): subscription homepass_id updated to target HomePass, previous_homepass_id snapshot retained, optional relocation fee charged per BIL-CFG-01 BillableEvent, physical work completed at both source and target via WO-01 SHIFTING flow (or self-move), FUL-07 bookkeeping updated on both HomePasses, source HomePass network-side de-provisioned (FUL-04), target HomePass provisioned for the current Package (FUL-03), last_relocated_at master field updated, customer notified.

For non-IMMEDIATE effectiveTiming, the subscription remains ACTIVE on source during the wait. A second pending-future relocation is rejected; any state-affecting operation during the wait cancels the pending relocation.

What this DD owns

1. subscription_relocation_config **table** — per-operator config.
2. **Trigger taxonomy** — 2 triggers with role gating.
3. **Target-HomePass validation rules** — sellable, service-family compatibility for current Package, port-provisioned, status active.
4. equipmentPath **taxonomy + auto-inference rules**.
5. disconnectMode **taxonomy + resolution rules**.
6. effectiveTiming **taxonomy + non-IMMEDIATE timer + cancellation paths**.
7. **WO-01 SHIFTING integration sequencing** — when to create the WO, what context to pass, which events to park on.
8. **Trigger + cancel REST endpoints**.
9. **Domain Kafka events** — SubscriptionRelocationScheduled, SubscriptionRelocated, SubscriptionRelocationRejected, SubscriptionRelocationCancelled.

What this DD does NOT own

- **BPMN files** — per-operator satellites.
- **Worker implementations** — DD_SUB-WF-FRAMEWORK-01-WORKERS.
- **Subscription state machine** — SUB-LM-01.
- **Package catalog** — SIP-01.
- **Service catalog** — PLM-CFG-01.
- **HomePass row mutations** — RLM-CFG-01. Read-only.
- **WO state machine, BPMN, contractor model, finalization mechanics** — WO-01-FRAMEWORK + WO-01-FLOW-SHIFTING.
- **Equipment binding mutations** — handled internally by WO-01 SHIFTING flow (for tech-dispatch path) or remain unchanged (for self-move path).
- **Relocation-fee math + BillableEvent definitions** — BIL-01 + BIL-CFG-01.
- **Source-side service teardown + target-side activation + per-service-class provisioner fan-out** — FUL-03 + FUL-04.

- HomePass bookkeeping (CLAIM / RELEASE) — FUL-07.

- Customer notification template + channel — NOT-01.

Regional flexibility

Lever	Examples
BPMN file	One per operator: Subscription_Relocation_<operator>.bpmn. Operator-specific branching, payment-timeout durations, notification policy live in the BPMN.
Drools package rules.subscription.relocation.<operator>	Operator-specific validation thresholds, equipment-path inference matrix, disconnect-mode defaults.
subscription_relocation_config row	Per-operator defaults: self-service enablement, effective-timing default, scheduled-window length, customer-notification policy.
WO-01 FLOW-SHIFTING (referenced)	Per-operator job-type codes, equipment-recovery policy, contractor-payment side effects, topology-capture vs topology-confirm at finalize.

2. Tables overview

This DD owns one table: subscription_relocation_config (per-operator config).

Cross-operation audit anchor: subscription_operation (framework-owned).

Subscription master fields updated (owned by SUB-LM-01): homepass_id, previous_homepass_id, last_relocated_at, last_status_changed_at. current_transition_type = RELOCATION during PENDING_RELOCATION.

3. Schemas

3.1 subscription_relocation_config

One row per operator.

```
CREATE TABLE subscription_relocation_config (  
  operator_code TEXT PRIMARY KEY,  
  customer_self_service_enabled BOOLEAN NOT NULL DEFAULT FALSE,  
  pay_first_required BOOLEAN NOT NULL DEFAULT TRUE,  
  default_effective_timing TEXT NOT NULL DEFAULT 'SCHEDULED_AT',  
  max_future_scheduled_days INTEGER NOT NULL DEFAULT 60,  
  default_disconnect_mode_keep_existing TEXT NOT NULL DEFAULT 'TECH_DISPATCH',  
  customer_notification_enabled BOOLEAN NOT NULL DEFAULT TRUE,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_by TEXT NOT NULL,  
  CHECK (default_effective_timing IN ('IMMEDIATE', 'END_OF_CURRENT_CYCLE', 'SCHEDULED_AT')),  
  CHECK (default_disconnect_mode_keep_existing IN ('TECH_DISPATCH', 'CUSTOMER_SELF_MOVE')),  
  CHECK (max_future_scheduled_days > 0)  
);
```

Column	Notes
customer_self_service_enabled	When true, the CUSTOMER_REQUESTED_RELOCATION trigger may be initiated from the customer portal. Operator policy.
pay_first_required	When true AND BIL-01 returns positive totals.net: pay-first gate at COMMIT time.
default_effective_timing	Default effectiveTiming applied when request omits. Operator policy.
max_future_scheduled_days	Maximum window for SCHEDULED_AT. scheduledAt. Operator policy.
default_disconnect_mode_keep_existing	Default disconnectMode for KEEP_EXISTING when request omits. NEW_INSTALL always forces TECH_DISPATCH regardless of this column.
customer_notification_enabled	When true, notification at acceptance AND at commit. NOT-01 owns template, channel, language.

WO timeouts are NOT configured here — they live in WO-01's framework config (wo_kind_config(operator_code, kind='SHIFTING').default_timeout_days) and govern the entire SHIFTING WO. Per DD_WO-01-CONTRACT-FOR-SUB-WF §1.7.

Sample data:

operator_code	customer_self_service_enabled	pay_first_required	default_effective_timing	max_future_scheduled_days	default_disconnect_mode_keep_existing	customer_notification_enabled
WIK	true	true	SCHEDULED_AT	60	TECH_DISPATCH	true
WUG	false	true	SCHEDULED_AT	60	TECH_DISPATCH	true
WTZ	false	true	SCHEDULED_AT	60	TECH_DISPATCH	true
YASSN	false	true	SCHEDULED_AT	60	TECH_DISPATCH	true

(Operators initialize their own row; values illustrative. Per-operator deep-dives may add columns as flexibility needs surface.)

4. Rules

4.1 State precondition + transition

Rule	Description
R-SUB-WF-RELOCATION-01-S-1	Relocation requires subscription's <code>status_code</code> = <code>ACTIVE</code> at request time AND at commit time. Other statuses return 409 <code>INVALID_STATE_FOR_RELOCATION</code> . For scheduled-future, commit-time re-validation may fail if the subscription was paused or terminated during the wait (per R-EF-5).
R-SUB-WF-RELOCATION-01-S-2	The transition at commit is <code>ACTIVE</code> → <code>PENDING_RELOCATION</code> → <code>ACTIVE</code> . <code>PENDING_RELOCATION</code> is entered only during the commit window. For scheduled-future, subscription stays <code>ACTIVE</code> during the wait — <code>PENDING_RELOCATION</code> entered when timer fires.
R-SUB-WF-RELOCATION-01-S-3	Per framework R-FW-1, relocation <code>COMMIT</code> is mutually exclusive with other state-affecting operations. A scheduled-future relocation in wait state DOES allow other operations to fire (which cancel the pending relocation per R-EF-5).
R-SUB-WF-RELOCATION-01-S-4	The subscription's <code>current_transition_type</code> is set to <code>RELOCATION</code> during <code>PENDING_RELOCATION</code> . <code>current_transition_reason_code</code> is set to <code>relocationTrigger</code> . NOT set during scheduled-future wait.
R-SUB-WF-RELOCATION-01-S-5	Restrictions on the subscription (<code>active_restrictions[]</code>) are preserved post-relocation. The new HomePass inherits the restriction set. Operator policy may decide to clear or re-evaluate; this DD does not.
R-SUB-WF-RELOCATION-01-S-6	At most one pending-future relocation per subscription. A second non- <code>IMMEDIATE</code> request returns 409 <code>PENDING_RELOCATION_ALREADY_SCHEDULED</code> . An <code>IMMEDIATE</code> request while a pending-future exists returns 409 (caller must cancel pending one first).

4.2 Trigger taxonomy + role gating

Rule	Description
R-SUB-WF-RELOCATION-01-RT-1	2 triggers: <code>CUSTOMER_REQUESTED_RELOCATION</code> and <code>ADMIN_RELOCATION</code> . Carried in request as <code>relocationTrigger</code> . Unknown values return 400 <code>UNKNOWN_RELOCATION_TRIGGER</code> .
R-SUB-WF-RELOCATION-01-RT-2	Role-trigger gating: <code>CUSTOMER_REQUESTED_RELOCATION</code> requires <code>CUSTOMER_SELF_SERVICE</code> OR <code>CALL_CENTER</code> OR <code>SUBSCRIPTION_OPERATOR</code> ; <code>ADMIN_RELOCATION</code> requires <code>SUBSCRIPTION_ADMIN</code> OR <code>SUBSCRIPTION_OPERATOR</code> . Mismatched role returns 403.
R-SUB-WF-RELOCATION-01-RT-3	<code>CUSTOMER_SELF_SERVICE</code> role allowed only when <code>subscription_relocation_config.customer_self_service_enabled</code> = <code>true</code> . Else 403.

4.3 Target-HomePass validation

Rule	Description
R-SUB-WF-RELOCATION-01-TH-1	<code>targetHomepassId</code> MUST reference a HomePass with <code>status_code.is_sellable</code> = <code>true</code> in RLM-CFG-01. Otherwise 400 <code>TARGET_HOMEPASS_NOT_SELLABLE</code> .
R-SUB-WF-RELOCATION-01-TH-2	<code>targetHomepassId</code> MUST differ from source <code>homepass_id</code> . Same-HomePass returns 400 <code>SAME_HOMEPASS</code> .

Rule	Description
R-SUB-WF-RELOCATION-01-TH-3	The target HomePass MUST be physically capable of carrying every service family required by the current Package. For each Service in the Package's package_composition, the Service's service_management_key MUST be present in targetHomepass.service_management_endpoints keys. If absent, return 400 CURRENT_PACKAGE_NOT_AVAILABLE_AT_TARGET_HOMEPASS with missing keys (caller may need to use MIGRATION instead).
R-SUB-WF-RELOCATION-01-TH-4	For each required service_management_key, the resolved endpoint's port MUST NOT be null. Null port acceptable for effectiveTiming = SCHEDULED_AT IF the SHIFTING WO will provision the port at reconnect-finalize; commit-time re-validation enforces. Null port for IMMEDIATE returns 400 TARGET_HOMEPASS_PORT_NOT_PROVISIONED.

4.4 Equipment path resolution

Rule	Description
R-SUB-WF-RELOCATION-01-EQ-1	equipmentPath ∈ {KEEP_EXISTING, NEW_INSTALL}. If the request supplies a value, Drools validates it; otherwise Drools auto-infers from network-tech compatibility.
R-SUB-WF-RELOCATION-01-EQ-2	Auto-inference: if sourceHomepass.networkPath.rootNodeType == targetHomepass.networkPath.rootNodeType (same network tech), inferred = KEEP_EXISTING. Otherwise inferred = NEW_INSTALL. The operator's compatibility matrix in Drools may declare additional same-tech-but-incompatible pairs → forced to NEW_INSTALL.
R-SUB-WF-RELOCATION-01-EQ-3	Request override: if the request supplies equipmentPath, Drools validates compatibility. Request KEEP_EXISTING with incompatible network-tech returns 400 EQUIPMENT_INCOMPATIBLE_WITH_TARGET. Request NEW_INSTALL always accepted.
R-SUB-WF-RELOCATION-01-EQ-4	The resolved equipmentPath is carried in flowContext to WO-01 at WO creation. The SHIFTING flow uses it to drive its internal equipment-handling logic (KEEP_EXISTING → tech transports existing equipment; NEW_INSTALL → tech recovers old + installs new). Equipment binding mutations happen inside WO-01.
R-SUB-WF-RELOCATION-01-EQ-5	The resolved equipmentPath is persisted to the Kafka event payload + the framework's subscription_operation audit row.

4.5 Disconnect mode resolution

Rule	Description
R-SUB-WF-RELOCATION-01-DM-1	disconnectMode ∈ {TECH_DISPATCH, CUSTOMER_SELF_MOVE}. Resolution: (a) if equipmentPath = NEW_INSTALL, force TECH_DISPATCH; (b) if equipmentPath = KEEP_EXISTING and request supplies, use it; (c) if request omits, default from subscription_relocation_config.default_disconnect_mode_keep_existing. Request CUSTOMER_SELF_MOVE with NEW_INSTALL returns 400 DISCONNECT_MODE_INCOMPATIBLE_WITH_EQUIPMENT_PATH.
R-SUB-WF-RELOCATION-01-DM-2	When disconnectMode = TECH_DISPATCH: the workflow creates a SHIFTING WO via WO-01's POST /api/work-orders (per DD_WO-01-CONTRACT-FOR-SUB-WF §1.3), then parks on WorkOrderFinalized (per §1.2 of that contract). The SHIFTING flow internally handles disconnect-then-reconnect, equipment recovery, topology resolution, account sub-status updates.
R-SUB-WF-RELOCATION-01-DM-3	When disconnectMode = CUSTOMER_SELF_MOVE: NO SHIFTING WO is created. The workflow logs a self-move acknowledgement step (no field intervention). The customer disconnects, transports, and reconnects their own equipment at the target. Equipment bindings remain unchanged on the subscription. KE WIK v1.1: path enabled but conservative defaults route most customers through TECH_DISPATCH.
R-SUB-WF-RELOCATION-01-DM-4	The resolved disconnectMode is persisted to the Kafka event payload + the framework's subscription_operation audit row.

4.6 Billing

Rule	Description
R-SUB-WF-RELOCATION-01-B-1	Workflow calls bil01-emit-intent with intentCode = RELOCATION_INTENT. Context: {sourceHomepassId, sourceFranchiseId, sourceNetworkRootType, targetHomepassId, targetFranchiseId, targetNetworkRootType, sameFranchise, equipmentPath, disconnectMode, relocationTrigger, scheduledCommitAt?}. BIL-01 resolves matching BillableEvents per BIL-CFG-01 catalog (e.g., RELOCATION_FEE_SAME_FRANCHISE, RELOCATION_FEE_CROSS_FRANCHISE, RELOCATION_FEE_NEW_INSTALL, EQUIPMENT_REPLACEMENT_FEE).
R-SUB-WF-RELOCATION-01-B-2	No proration delta (same Package). The current cycle continues uninterrupted at the current Package price. The only money movement is the optional relocation-related fees.
R-SUB-WF-RELOCATION-01-B-3	If pay_first_required = true AND BIL-01's totals.net > 0: pay-first gate at COMMIT time. The workflow waits for payment - confirmed before creating the SHIFTING WO (or before proceeding past the self-move acknowledgement). Payment timeout (operator-config; e.g., 7 days) → cancel the relocation.
R-SUB-WF-RELOCATION-01-B-4	If totals.net = 0 (no fee), pay-first gate is skipped.

4.7 Fulfillment sequence

Rule	Description
R-SUB-WF-RELOCATION-01-F-1	Fulfillment sequence (in order, during commit window): (a) FUL-04 SUSPEND_SERVICE at source HomePass — deactivate per-service-class provisioning; (b) FUL-07 RELEASE on source HomePass — close homepass_subscription_binding, decrement ATP, release port bindings if AT_INSTALLATION; (c) physical work — if disconnectMode = TECH_DISPATCH: create SHIFTING WO via POST /api/work-orders with flowContext={equipmentPath, disconnectMode, sourceHomepassId, targetHomepassId}, park on WorkOrderFinalized; if CUSTOMER_SELF_MOVE: log self-move acknowledgement and proceed; (d) FUL-03 ACTIVATE_SERVICE at target HomePass with equipment bindings (from WorkOrderFinalized.finalEquipmentBindings for tech-dispatch; from pre-existing bindings for self-move); (e) FUL-07 CLAIM on target HomePass — open new homepass_subscription_binding, increment ATP, capture port bindings if AT_INSTALLATION; (f) commit master fields (homepass_id, previous_homepass_id, last_relocated_at).
R-SUB-WF-RELOCATION-01-F-2	Atomic-switch: subscription stays ACTIVE on source HomePass until step (d) succeeds. If any step before (e) fails, the workflow reverts: FUL-04 RESUME_SERVICE at source (best-effort), FUL-07 CLAIM back on source (best-effort if previously RELEASED), revert state to ACTIVE, emit SubscriptionRelocationRejected. For TECH_DISPATCH path, if the SHIFTING WO already partially completed (disconnect-phase done, reconnect-phase failed), operations alert fires for manual recovery — equipment is in tech custody or installed at target without subscription activation.
R-SUB-WF-RELOCATION-01-F-3	WO timeout: handled by WO-01 framework (wo_kind_config(kind=SHIFTING).default_timeout_days). On timeout, WO-01 emits WorkOrderTimedOut. The workflow's catch fires; runs F-2 compensation; emits SubscriptionRelocationRejected with rejectionReasonCode = WO_TIMEOUT.
R-SUB-WF-RELOCATION-01-F-4	WO cancellation: if the dispatcher cancels the SHIFTING WO mid-flow (operations intervention), WO-01 emits WorkOrderCancelled. The workflow's catch fires; runs F-2 compensation; emits SubscriptionRelocationRejected with rejectionReasonCode = WO_CANCELLED_BY_DISPATCHER.
R-SUB-WF-RELOCATION-01-F-5	HomePass row is NOT mutated by this workflow. RLM-CFG-01 is read-only from this workflow's perspective. WO-01 (SHIFTING flow) and FUL-07 own their respective mutations independently.

4.8 Effective timing

Rule	Description
R-SUB-WF-RELOCATION-01-EF-1	Request supplies effectiveTiming ∈ {IMMEDIATE, END_OF_CURRENT_CYCLE, SCHEDULED_AT}. If absent: default from subscription_relocation_config.default_effective_timing. Unknown returns 400 UNKNOWN_EFFECTIVE_TIMING.

Rule	Description
R-SUB-WF-RELOCATION-01-EF-2	When effectiveTiming = SCHEDULED_AT: request MUST supply scheduledAt. Missing → 400 SCHEDULED_AT_MISSING. Must be strictly future and within max_future_scheduled_days. Past or beyond-window → 400 SCHEDULED_AT_OUT_OF_BOUNDS.
R-SUB-WF-RELOCATION-01-EF-3	For non-IMMEDIATE timings: BPMN parks on Camunda intermediate timer. subscription_operation.current_state = SCHEDULED. Trigger endpoint returns 202 with scheduledCommitAt populated. SubscriptionRelocationScheduled emitted on Kafka. At timer fire, BPMN re-validates and enters PENDING_RELOCATION + commit sequence.
R-SUB-WF-RELOCATION-01-EF-4	Cancellation paths during wait: (a) explicit DELETE /pending-relocation/{operationId} — stops timer, emits SubscriptionRelocationCancelled, closes operation as CANCELLED; (b) any state-affecting operation on same subscription — framework's external-cancel-requested mechanism cancels with cancellationReasonCode = SUPERSEDED_BY_<otherOperationKind>.
R-SUB-WF-RELOCATION-01-EF-5	At commit time (after wait), workflow re-validates all preconditions including target-HomePass + equipment-path resolution. Fails with rejectionStage = COMMIT_REVALIDATION.
R-SUB-WF-RELOCATION-01-EF-6	Cancellation during COMMIT WINDOW (after timer fire, past source suspend) is NOT allowed via DELETE — atomic-switch in progress. Cancel attempts return 409 OPERATION_IN_COMMIT_WINDOW. The dispatcher may force-cancel the SHIFTING WO independently via WO-01's cancel endpoint; the workflow then handles the unexpected WorkOrderCancelled event per F-4.
R-SUB-WF-RELOCATION-01-EF-7	Pay-first gate at COMMIT time, not request time. For non-IMMEDIATE relocations, customer wallet / card only touched at commit.

5. REST API

5.1 Trigger relocation — scheduled (the typical case)

```
POST /api/subscriptions/{subscription_id}/relocate HTTP/1.1
Authorization: Bearer <jwt>
Content-Type: application/json
Idempotency-Key: relocate-sub_01HZ_KAMAU-hp_LAVI_073-20260601
```

```
{
  "operatorCode":      "WIK",
  "relocationTrigger": "CUSTOMER_REQUESTED_RELOCATION",
  "targetHomepassId":  "hp_01HZ_LAVI_073",
  "equipmentPath":     null,
  "disconnectMode":    null,
  "effectiveTiming":    "SCHEDULED_AT",
  "scheduledAt":        "2026-06-01T09:00:00+03:00",
  "notes":              "Customer moving to new apartment in Lavington on June 1",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole":  "CALL_CENTER",
  "initiatingWorkflowKind": "BO_UI",
  "initiatingWorkflowRef": "cc-ticket-2026-05-15-3201"
}
```

→ 202 Accepted

```
{
  "operationId":      "subop_01HZ_KAMAU_RELOCATE",
  "operatorCode":      "WIK",
  "subscriptionId":    "sub_01HZ_KAMAU_FIBER",
  "operationKind":      "RELOCATION",
  "relocationTrigger": "CUSTOMER_REQUESTED_RELOCATION",
  "resolvedEquipmentPath": "KEEP_EXISTING",
  "resolvedDisconnectMode": "TECH_DISPATCH",
  "resolvedEffectiveTiming": "SCHEDULED_AT",
  "scheduledCommitAt":  "2026-06-01T09:00:00+03:00",
  "bpmnProcessKey":     "sub-relocate-wik",
  "bpmnProcessInstanceId": "pi_01HZ_KAMAU_RELOCATE",
  "currentState":       "SCHEDULED",
  "startedAt":          "2026-05-15T14:00:00+03:00"
}
```

(Request omitted equipmentPath and disconnectMode; Drools auto-inferred KEEP_EXISTING based on same network tech, and TECH_DISPATCH from operator config default.)

5.2 Trigger relocation — IMMEDIATE

```
POST /api/subscriptions/{subscription_id}/relocate HTTP/1.1
{
  "operatorCode":      "WIK",
  "relocationTrigger":  "ADMIN_RELOCATION",
  "targetHomepassId":  "hp_01HZ_LAVI_073",
  "equipmentPath":     "NEW_INSTALL",
  "effectiveTiming":    "IMMEDIATE",
  "initiatingActorRole": "SUBSCRIPTION_ADMIN"
}

→ 202 Accepted
{
  "resolvedEquipmentPath": "NEW_INSTALL",
  "resolvedDisconnectMode": "TECH_DISPATCH",
  "resolvedEffectiveTiming": "IMMEDIATE",
  "scheduledCommitAt": null,
  "currentState": "VALIDATING"
}
```

(disconnectMode omitted; forced to TECH_DISPATCH because equipmentPath = NEW_INSTALL.)

5.3 Errors

```
→ 400 Bad Request
{
  "code": "CURRENT_PACKAGE_NOT_AVAILABLE_AT_TARGET_HOMEPASS",
  "currentPackageRef": "pkg_01HX_INET_VOIP_IPTV_RES",
  "targetHomepassId": "hp_01HZ_KIBE_010",
  "missingServiceManagementKeys": ["iptv_multicast"],
  "guidance": "Target HomePass does not support all service families required by the current Package."
}

→ 400 Bad Request
{
  "code": "EQUIPMENT_INCOMPATIBLE_WITH_TARGET",
  "sourceNetworkRootType": "<source-tech>",
  "targetNetworkRootType": "<target-tech>",
  "requestedEquipmentPath": "KEEP_EXISTING",
  "guidance": "Source equipment cannot be re-homed on the target network. Use equipmentPath=NEW_INSTALL."
}

→ 400 Bad Request
{
  "code": "DISCONNECT_MODE_INCOMPATIBLE_WITH_EQUIPMENT_PATH",
  "equipmentPath": "NEW_INSTALL",
  "requestedDisconnectMode": "CUSTOMER_SELF_MOVE",
  "guidance": "NEW_INSTALL requires TECH_DISPATCH so old equipment can be physically recovered for return-to"
}
```

5.4 Cancel pending-future relocation

```
DELETE /api/subscriptions/{subscription_id}/pending-relocation/{operation_id} HTTP/1.1
{
  "operatorCode":      "WIK",
  "cancellationReasonCode": "CUSTOMER_CHANGED_MIND",
  "initiatingActorUserId": "cc-agent-wik-007",
  "initiatingActorRole": "CALL_CENTER"
}

→ 200 OK
{
  "operationId": "subop_01HZ_KAMAU_RELOCATE",
  "cancellationReasonCode": "CUSTOMER_CHANGED_MIND",
  "cancelledAt": "2026-05-20T16:00:00+03:00",
  "subscriptionStaysOn": {"homepassId": "hp_01HZ_KAREN_017"},
  "finalState": "CANCELLED"
}
```

In-commit-window cancel:

```
→ 409 Conflict
{
  "code": "OPERATION_IN_COMMIT_WINDOW",
  "currentState": "AWAITING_WO_FINALIZED",
  "guidance": "Relocation is in the atomic-switch commit window. Dispatcher may force-cancel the SHIFTING WO via WO-01"
}
```

6. Kafka events emitted

Event	Topic	Fires on	Consumers
SubscriptionRelocationScheduled	sophix.subscription.subscription.relocationscheduled	Acceptance of non-IMMEDIATE timing	NOT-01, CVM, EM-04
SubscriptionRelocated	sophix.subscription.subscription.relocated	Successful commit	NOT-01, BIL-02 (cycle accrual at new HomePass), CVM (move analytics), EM-04 reporting
SubscriptionRelocationCancelled	sophix.subscription.subscription.relocationcancelled	Pending-future cancelled	NOT-01, CVM, EM-04
SubscriptionRelocationRejected	sophix.subscription.subscription.relocationrejected	Workflow failure (validation, billing, WO timeout, WO cancelled, fulfillment, commit-time revalidation)	NOT-01, ICN-01 ops alert, EM-04

Sample SubscriptionRelocationScheduled:

```
{
  "eventType": "SubscriptionRelocationScheduled",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp": "2026-05-15T14:00:01+03:00",
  "payload": {
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "customerId": "cust_01HZ_KAMAU",
    "operationId": "subop_01HZ_KAMAU_RELOCATE",
    "relocationTrigger": "CUSTOMER_REQUESTED_RELOCATION",
    "sourceHomepassId": "hp_01HZ_KAREN_017",
    "sourceFranchiseId": "frn_NRB_KAREN_RES",
    "sourceNetworkRootType": "<source-root-type>",
    "targetHomepassId": "hp_01HZ_LAVI_073",
    "targetFranchiseId": "frn_NRB_LAVI_RES",
    "targetNetworkRootType": "<target-root-type>",
    "sameFranchise": false,
    "resolvedEquipmentPath": "KEEP_EXISTING",
    "resolvedDisconnectMode": "TECH_DISPATCH",
    "resolvedEffectiveTiming": "SCHEDULED_AT",
    "scheduledCommitAt": "2026-06-01T09:00:00+03:00",
    "initiatingActorUserId": "cc-agent-wik-007",
    "initiatingActorRole": "CALL_CENTER"
  }
}
```

Sample SubscriptionRelocated:

```
{
  "eventType": "SubscriptionRelocated",
  "aggregateId": "sub_01HZ_KAMAU_FIBER",
  "timestamp": "2026-06-01T10:35:42+03:00",
  "payload": {
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_KAMAU_FIBER",
    "operationId": "subop_01HZ_KAMAU_RELOCATE",
    "relocationTrigger": "CUSTOMER_REQUESTED_RELOCATION",
    "sourceHomepassId": "hp_01HZ_KAREN_017",
    "targetHomepassId": "hp_01HZ_LAVI_073",
    "sameFranchise": false,
    "resolvedEquipmentPath": "KEEP_EXISTING",
    "resolvedDisconnectMode": "TECH_DISPATCH",
    "resolvedEffectiveTiming": "SCHEDULED_AT",
    "scheduledCommitAt": "2026-06-01T09:00:00+03:00",
    "actualCommitAt": "2026-06-01T10:35:42+03:00",
    "shiftingWoId": "wo_01HZ_KAMAU_SHIFT_3201",
    "currentPackageRef": "pkg_01HX_INET_100M_VOIP_RES",
    "currentPackageVersionId": "pkv_01HX_INET_100M_V3",
    "relocationFeesNet": 500.00,
    "currency": "KES",
    "initiatingActorUserId": "cc-agent-wik-007",
    "initiatingActorRole": "CALL_CENTER"
  }
}
```

7. Cross-DD coordination

7.1 Upstream callers

Caller	relocationTrigger	Role
Customer portal	CUSTOMER_REQUESTED_RELOCATION	CUSTOMER_SELF_SERVICE (subject to config)
Call Center	CUSTOMER_REQUESTED_RELOCATION	CALL_CENTER
BO admin	ADMIN_RELOCATION	SUBSCRIPTION_ADMIN or SUBSCRIPTION_OPERATOR

7.2 Sibling modules

Module	Direction	Contract
SUB-WF-FRAMEWORK-01 (parent)	This DD composes framework primitives	subscription_operation row + BPMN start
SUB-WF-FRAMEWORK-01-WORKERS	This DD's BPMNs reference workers by topic	Standard framework workers + wo-create-shifting + wo-cancel
SUB-LM-01	Workers write status + master fields atomically	homepass_id + previous_homepass_id + last_relocated_at
RLM-CFG-01	Workers read source + target HomePass	service_management_endpoints, status_code, network_path root node type, franchise
SIP-01	Workers read current Package	Service composition
PLM-CFG-01	Workers read service definitions	service_management_key for HomePass capability check
BIL-01 v1.1	bil01-emit-intent worker	RELOCATION_INTENT — relocation fees per BIL-CFG-01
BIL-CFG-01 v1.0	BillableEvent catalog	Relocation fee definitions per operator
FUL-04 v1.1	ful-call-suspend worker	SUSPEND_SERVICE at source HomePass
FUL-03 v1.1	ful-call-activate worker	ACTIVATE_SERVICE at target HomePass
FUL-07 v1.0	ful-call-release-homepass + ful-call-claim-homepass workers	HomePass bookkeeping (RELEASE source, CLAIM target)
WO-01 (FRAMEWORK + FLOW-SHIFTING)	wo-create-shifting worker (when disconnectMode = TECH_DISPATCH)	Per DD_WO-01-CONTRACT-FOR-SUB-WF — POST /api/work-orders with kind=SHIFTING; BPMN parks on WorkOrderFinalized
NOT-01	notification-send worker	Customer notification per operator config
BIL-02 / CVM / EM-04	Consume Kafka events	Cycle accrual at new HomePass, move analytics, reporting

7.3 Build order

This DD deploys after:

1. SUB-WF-FRAMEWORK-01 (parent + WORKERS) v1.0
2. SUB-LM-01 v1.0
3. SIP-01 v1.0, PLM-CFG-01 v1.0, RLM-CFG-01 v1.0
4. FUL-03 v1.1, FUL-04 v1.1, FUL-07 v1.0
5. BIL-01 v1.1, BIL-CFG-01 v1.0
6. WO-01 framework + FLOW-SHIFTING v1.0
7. DD_WO-01-CONTRACT-FOR-SUB-WF v1.0 (integration surface)
8. At least one FLOW satellite per operator (FLOW-WIK v1.1 ships alongside this parent)

7.4 Stub debt + open coordination items

- **WUG / WTZ / YASSN FLOW satellites** — pending operator deep dives. Each operator deploys its own Subscription_Relocation_<operator>.bpmn referencing its own Drools KJAR + subscription_relocation_config row + WO-01 SHIFTING flow variant.
- **Operator-specific job-type code mapping** — operator's WO-01 Drools KJAR (per DD_WO-01-CONTRACT-FOR-SUB-WF §1.4) handles the mapping from (kind=SHIFTING, sourceNetworkRootType, equipmentPath) to the operator's specific job-type code.

B — Current TZ

Status: To be filled in after codebase cartography review.

A relocation flow likely exists in the TZ codebase. Specific structural details — whether existing code uses a single SHIFTING WO with phases or separate WOs, whether equipment recovery is in-flow or async, whether atomic-switch service continuity is preserved — require source-code review to document accurately.